

Министерство науки и высшего образования Российской Федерации ФЕДЕРАЛЬНОЕ
ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»
(Университет ИТМО)

Факультет прикладной информатики
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии

О Т Ч Е Т
по предмету Методы оптимизации
Тема задания: «Лабораторная №1. Симплекс метод»

Обучающийся: Левахин Лев Александрович, 343730, К3340
Вариант: 13

Санкт-Петербург,
2026

Задание 1

Ручное решение

Левых Лев 343730 поток 1.2 вариант 13

$$Z = 3X_1 + X_2 + 2X_3 + 4X_4 \rightarrow \max$$

$$\begin{cases} 2X_1 + X_2 + X_3 \leq 8 \\ X_1 + X_3 + X_4 = 6 \\ X_2 + X_4 \geq 4 \end{cases} \quad X_1, X_2, X_3, X_4 \geq 0$$

$$W = -Z = -3X_1 - X_2 - 2X_3 - 4X_4 \rightarrow \min \quad (Z \rightarrow \max \leftrightarrow -Z \rightarrow \min)$$

→ Избавляемся от нер-в

Введем X_5 и $X_6 \geq 0$. $\Rightarrow$ Канонический вид

$$\begin{cases} 2X_1 + X_2 + X_3 + X_5 = 8 \\ X_1 + X_3 + X_4 = 6 \\ X_2 + X_4 - X_6 = 4 \end{cases} \quad W = -3X_1 - X_2 - 2X_3 - 4X_4 \rightarrow \min$$

→ Составим базис. Ищем перем., которые не входят в другие и без коэф.

в ① X_5 во ② и ③ - нет такой

$\Rightarrow$ вводим искусств. X_7 и $X_8 \geq 0$

→ Вспомогат. задача

$$W' = X_7 + X_8 \rightarrow \min \quad \begin{cases} 2X_1 + X_2 + X_3 + X_5 = 8 \\ X_1 + X_3 + X_4 + X_7 = 6 \\ X_2 + X_4 - X_6 + X_8 = 4 \end{cases}$$

$\Rightarrow$ Базисные перем. X_5, X_7, X_8

Сначала найдем решение вспом. задачи $W'_{\min} \geq 0$, потом убираем X_7 и X_8 и переходим к осн. задаче

$$\begin{aligned} X_7 &= 6 - X_1 - X_3 - X_4 \rightarrow W' = 6 - X_1 - X_3 - X_4 + 4 - X_2 - X_4 + X_6 = \\ X_8 &= 4 - X_2 - X_4 + X_6 \end{aligned}$$

→ Состав. симплекс табл. для вспом. задачи.

Базис	X_1	X_2	X_3	X_4	X_5	X_6	X_7	X_8	Рес.
X_5	2	1	1	0	1	0	0	0	8
X_7	1	0	1	1	0	0	1	0	6
X_8	0	1	0	1	0	-1	0	1	4
W'	-1	-1	-2	-2	0	1	0	0	-10

Разрешающий столбец: $\min\{W'\} = -2 \Rightarrow j^* = 3 (X_3)$

строка: $\min\left\{\frac{2}{1}, \frac{6}{1}\right\} = 2 \Rightarrow i^* = 2 (X_7)$

элемент: 1 $\rightarrow X_7 \rightarrow X_3$

Перерасчет:

Базис	X_1	X_2	X_3	X_4	X_5	X_6	X_7	X_8	Реш
X_5	1	1	0	-1	1	0	-1	0	2
X_3	1	0	1	1	0	0	1	0	6
X_6	0	1	0	1	0	-1	0	1	4
W'	1	-1	0	0	0	1	2	0	2

Разр. столбец: $\min\{W'\} = -1 \Rightarrow X_2$

строка: $\min\left\{\frac{2}{1}, \frac{4}{1}\right\} = 2 \Rightarrow X_5$

$\Rightarrow \geq 1. 1 \rightarrow X_5 \rightarrow X_2$

Перерасчет:

Базис	X_1	X_2	X_3	X_4	X_5	X_6	X_7	X_8	Реш
X_2	1	1	0	-1	1	0	-1	0	2
X_3	1	0	1	1	0	0	1	0	6
X_6	-1	0	0	2	-1	-1	1	1	2
W'	2	0	0	-1	1	1	1	0	4

Разр.

столбец: $-1 \Rightarrow X_4$

строка: $\min\left\{\frac{2}{2}, \frac{6}{1}, \frac{2}{1}\right\} = 1 \Rightarrow X_6$

$\geq 1. \div 2$

$X_6 \rightarrow X_4$

Перерасчет:

Базис	X_1	X_2	X_3	X_4	X_5	X_6	X_7	X_8	Реш
X_2	$\frac{1}{2}$	1	0	0	$\frac{1}{2}$	$-\frac{1}{2}$	$-\frac{1}{2}$	$\frac{1}{2}$	3
X_3	$\frac{3}{2}$	0	1	0	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2}$	$-\frac{1}{2}$	5
X_4	$-\frac{1}{2}$	0	0	1	$-\frac{1}{2}$	$-\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2}$	1
W'	$\frac{3}{2}$	0	0	0	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{3}{2}$	$\frac{1}{2}$	0

$\leftarrow$ нет отриц., $\Rightarrow$ ок

Т.к. $W' = 0$ то искусств. переи. можно считать $= 0$ $\Rightarrow$ всякая задача реш.

$\Rightarrow$ Переходим к основной задаче

Убрав X_7 и X_8 , базисные (X_2, X_3, X_4) , свободные (X_1, X_5, X_6)

$$X_2 = 3 - \frac{1}{2}X_1 - \frac{1}{2}X_5 + \frac{1}{2}X_6$$

$$X_3 = 5 - \frac{3}{2}X_1 - \frac{1}{2}X_5 - \frac{1}{2}X_6$$

$$X_4 = 1 + \frac{1}{2}X_1 + \frac{1}{2}X_5 + \frac{1}{2}X_6$$

$$W = -3X_1 - (3 - \frac{1}{2}X_1 - \frac{1}{2}X_5 + \frac{1}{2}X_6) - 2(5 - \frac{3}{2}X_1 - \frac{1}{2}X_5 - \frac{1}{2}X_6) - 4(1 + \frac{1}{2}X_1 + \frac{1}{2}X_5 + \frac{1}{2}X_6)$$

$$W = -\frac{3}{2}X_1 - \frac{1}{2}X_5 - \frac{3}{2}X_6 - 17 \rightarrow \min$$

→ Составим симплекс-таблицу для СМ задачи

	X_1	X_2	X_3	X_4	X_5	X_6	Рес
X_2	$\frac{1}{2}$	1	0	0	$\frac{1}{2}$	$-\frac{1}{2}$	3
X_3	$\frac{3}{2}$	0	1	0	$\frac{1}{2}$	$\frac{1}{2}$	5
X_4	$-\frac{1}{2}$	0	0	1	$-\frac{1}{2}$	$-\frac{1}{2}$	1
W	$-\frac{3}{2}$	0	0	0	$-\frac{1}{2}$	$-\frac{3}{2}$	-17

Рез. СМД: $\min\{W\} = -\frac{3}{2} \Rightarrow X_1$
 Сфера: $\min\left\{\frac{3}{1/2}, \frac{5}{1/2}, \frac{-10}{-1/2}\right\} = \frac{10}{3} \Rightarrow X_3$
 ЗЛ: $\frac{3}{2}$

$(X_3 \rightarrow X_1)$

X_2	0	1	$-\frac{1}{3}$	0	$\frac{1}{3}$	$-\frac{2}{3}$	$\frac{4}{3}$
X_1	1	0	$\frac{2}{3}$	0	$\frac{1}{3}$	$\frac{1}{3}$	$\frac{10}{3}$
X_4	0	0	$\frac{1}{3}$	1	$-\frac{1}{3}$	$-\frac{1}{3}$	$\frac{8}{3}$
W	0	0	1	0	0	-1	-12

Рез. симплекс: $\min\{W\} = -1 \Rightarrow X_6$
 Сфера: $\min\left\{\frac{10+3}{1/3}\right\} = 10 \Rightarrow X_1$
 ЗЛ: $\frac{1}{3}$

$(X_1 \rightarrow X_6)$

X_2	2	1	1	0	1	0	8
X_6	3	0	2	0	1	1	10
X_4	1	0	1	1	0	0	6
W	3	0	3	0	1	0	-2

← В симплекс нет отриц. ЗЛ-ов
 $\Rightarrow$ Рес. найдено

$$X_2 = 8, X_6 = 10, X_4 = 6, X_1 = 0, X_3 = 0, X_5 = 0 \Rightarrow Z = 3X_1 + X_2 + 2X_3 + 4X_4 = 8 + 24 = 32$$

$$Z = -W$$

$$\text{Ответ: } X^* = (0, 8, 0, 6); Z = 32$$

Все пояснения расписывал по ходу решения. Удалось прийти к ответу

$$X = (0, 8, 0, 6)$$

$$Z = 32$$

Задание 2

Решение в Excel

Чтобы решить задачу в Excel первым делом нужно составить таблицу

Переменные	x1	x2	x3	x4
Изм ячейки				
Целевая ф				

В этой таблице мы добавляем:

- Переменные, которые даны изначально
- Изменяемые ячейки, в которых алгоритм будет перебирать значения наших иксов
- И ниже целевая функция $=3*B2 + C2 + 2*D2 + 4*E2$

Далее мы составляем таблицу ограничений

Ограничения			
Огр1 ≤ 8	8	$\leq$	8
Огр2 $= 6$	6	$=$	6
Огр3 ≥ 4	14	$\geq$	4

В эту таблицу заносим в отдельные ячейки формулы, которые нам даны, то есть:

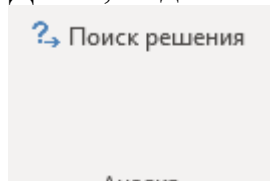
1) $=2*B2 + C2 + D2$

2) $= B2 + D2 + E2$

3) $=C2 + E2$

Так алгоритм в дальнейшем будет понимать, что ограничения соблюдаются

Далее, подготовив всё необходимое, нажмем поиск решения на вкладке Данные



Здесь мы выделяем подготовленные нами ячейки

Параметры поиска решения

Оптимизировать целевую функцию:

До: ☒ Максимум ☐ Минимум ☐ Значения:

Изменяя ячейки переменных:

В соответствии с ограничениями:

\$B\$2:\$E\$2 >= 0

\$B\$7 <= \$D\$7

\$B\$8 = \$D\$8

\$B\$9 >= \$D\$9

Добавить

Изменить

Удалить

Сбросить

Загрузить/сохранить

☒ Сделать переменные без ограничений неотрицательными

Выберите метод решения:

Параметры

Метод решения

Для гладких нелинейных задач используйте поиск решения нелинейных задач методом ОПГ, для линейных задач - поиск решения линейных задач симплекс-методом, а для негладких задач - эволюционный поиск решения.

Справка

Найти решение

Заккрыть

Так, алгоритм будет перебирать значения до тех пор, пока не найдёт самое оптимальное решение.

В результате получаем такой же ответ как и в решении руками. Значит решили верно!

Задание 3

Решение кодом

Что решает программа

Программа решает задачу линейного программирования симплекс-методом. На вход подаются коэффициенты целевой функции, матрица ограничений, правая часть и знаки ограничений. Программа сама приводит задачу к каноническому виду и находит оптимальное решение. Поддерживаются задачи на максимум и на минимум, а также свободные переменные которые могут быть любого знака.

Я решил написать класс SimplexSolver, который бы отвечал за все необходимые задачи по решению. Приведу несколько важных моментов оттуда

Приведение к каноническому виду

Метод `_build_canonical` делает сразу несколько вещей. Если в ограничении правая часть отрицательная мы умножаем всю строку на минус один и меняем знак неравенства на противоположный. Потом для каждого неравенства добавляется дополнительная переменная. Если знак `<=` то переменная прибавляется с коэффициентом плюс один, если знак `>=` то вычитается с коэффициентом минус один.

```
if signs[i] == '<=':
    k = sum(1 for s in signs[:i] if s in ('<=', '>='))
    row[self.n_new + k] = Fraction(1)
elif signs[i] == '>=':
    k = sum(1 for s in signs[:i] if s in ('<=', '>='))
    row[self.n_new + k] = Fraction(-1)
```

После этого мы получаем систему равенств и можем искать начальный базис.

Свободные переменные

Если переменная может быть любого знака мы делаем замену в методе `_apply_free_vars`. Вместо одной переменной появляется две, обе неотрицательные. Коэффициенты в целевой функции и в ограничениях дублируются с нужными знаками.

```
new_c.append(self.c_orig[j])
new_c.append(-self.c_orig[j])
for i in range(self.m):
    new_A[i].append(self.A_orig[i][j])
    new_A[i].append(-self.A_orig[i][j])
```

Дополнительно ведётся список `orig_map` где для каждой новой переменной хранится исходный индекс и знак. В конце по этой карте восстанавливаются значения исходных переменных.

Поиск начального базиса

Метод `_find_initial_basis` ищет столбцы которые выглядят как единичные. То есть в столбце ровно одна единица и все остальные элементы нули. Такая переменная сразу становится базисной.

```
ones = [i for i, v in enumerate(col) if v == 1]
others_zero = all(v == 0 or v == 1 for v in col)
if len(ones) == 1 and others_zero:
    i = ones[0]
    if i not in used_rows and basis[i] is None:
        basis[i] = j
```

Если для какой-то строки базис не нашёлся мы потом добавим туда искусственную переменную.

Вспомогательная задача

Метод `_build_auxiliary` добавляет искусственные переменные в те строки где нет базиса. Каждая искусственная переменная входит только в свою строку с коэффициентом один.

Целевая функция вспомогательной задачи это сумма всех искусственных переменных. Её тоже надо привести к текущему базису, то есть вычесть из неё строки где стоят искусственные переменные.

```
aux_c = [Fraction(0)] * self.total_vars
for j in art_indices:
    aux_c[j] = Fraction(1)
aux_b = Fraction(0)
for i in range(self.m):
    if self.basis[i] in art_indices:
        for j in range(self.total_vars):
            aux_c[j] -= self.A[i][j]
        aux_b -= self.b[i]
```

После решения вспомогательной задачи если минимум равен нулю значит допустимое решение есть и можно переходить к основной задаче. Если минимум больше нуля значит область допустимых решений пуста.

Симплекс итерации

Метод `_simplex` делает сами итерации. На каждом шаге ищется разрешающий столбец это самый отрицательный элемент в строке целевой функции. Потом ищется разрешающая строка по минимальному отношению правой части к положительным элементам столбца

```
j_star = None
```

```

min_val = Fraction(0)
for j in range(self.total_vars):
    if j in forbid:
        continue
    if c[j] < min_val:
        min_val = c[j]
        j_star = j

```

Если отрицательных элементов нет значит решение оптимально. Если в разрешающем столбце нет положительных элементов значит задача неограничена.

Пересчёт таблицы

Когда разрешающий элемент найден мы делим всю строку на него. Потом из остальных строк вычитаем новую разрешающую строку умноженную на нужный коэффициент. То же самое делаем со строкой целевой функции.

```

pivot = A[i_star][j_star]
A[i_star] = [v / pivot for v in A[i_star]]
b[i_star] = b[i_star] / pivot
for i in range(self.m):
    if i != i_star and A[i][j_star] != 0:
        factor = A[i][j_star]
        A[i] = [A[i][k] - factor * A[i_star][k] for k in range(self.total_vars)]
        b[i] = b[i] - factor * b[i_star]

```

Так продолжается пока в строке целевой функции есть отрицательные элементы

Главный метод solve

В итоге solve связывает всё вместе. Сначала проверяет полный ли базис. Если нет строит вспомогательную задачу и решает её. Потом убирает искусственные переменные из базиса если они там остались с нулём. Дальше приводит основную целевую функцию к текущему базису и запускает симплекс итерации. В конце собирает значения переменных и восстанавливает исходные переменные по карте orig_map

```

for j in range(self.n_new):
    orig_idx, sign = self.orig_map[j]
    x_orig[orig_idx] += sign * x_ext[j]

```

Результаты

Дальше я проверил результаты на своей задаче и на случайном примере. Ответы оказались правильными. Вот такой вывод:

```

=====
Задача по моему варианту 13
=====

Статус: optimal
  x1 = 0
  x2 = 8
  x3 = 0
  x4 = 6
Z (max) = 32

```

Получился универсальный солвер который сам приводит задачу к каноническому виду, сам строит вспомогательную задачу если нужно, поддерживает свободные переменные и не ломается на крайних случаях вроде неограниченной задачи или пустой области допустимых решений.

Полный листинг кода

```

from fractions import Fraction
from typing import List, Optional, Set

class SimplexSolver:
    """
    Универсальный симплекс-метод для задачи  $W \rightarrow \min$ 
    - Принимает задачу в общем виде, сам приводит к каноническому виду
    - Поддерживает свободные переменные (любого знака)
    - Работает с дробями для точности
    """

    def __init__(self, c, A, b, signs, maximize=True, free_vars=None):
        """
        c          - коэффициенты целевой функции
        A          - матрица ограничений
        b          - правая часть
        signs      - знаки ограничений: '<=', '>=', '='
        maximize   - True если исходно  $Z \rightarrow \max$ , False если  $W \rightarrow \min$ 
        free_vars  - множество индексов свободных переменных (могут быть
        любого знака)
        """
        self.n = len(c)
        self.m = len(A)

        factor = -1 if maximize else 1
        self.c_orig = [Fraction(factor * v) for v in c]
        self.A_orig = [[Fraction(v) for v in row] for row in A]
        self.b_orig = [Fraction(v) for v in b]
        self.signs_orig = list(signs)
        self.free_vars = set(free_vars) if free_vars else set()

        # имена переменных
        self.var_names: List[str] = []

```

```

# карта: индекс в новой задаче -> (исходный индекс, знак: +1 или -1)
self.orig_map = []

self._apply_free_vars()
self._build_canonical()

# -----
# Замена свободных переменных  $x_k = x_k^{+} - x_k^{-}$ 
# -----
def _apply_free_vars(self):
    """Заменяет свободные переменные разностью двух неотрицательных"""
    if not self.free_vars:
        # без замен: всё как есть
        self.n_new = self.n
        for i in range(self.n):
            self.orig_map.append((i, Fraction(1)))
        self.var_names_orig = [f"x_{i+1}" for i in range(self.n)]
        return

    new_c = []
    new_A = [[] for _ in range(self.m)]
    new_names = []

    for j in range(self.n):
        if j in self.free_vars:
            #  $x_j = x_j^{+} - x_j^{-}$ 
            new_c.append(self.c_orig[j]) # коэффициент при  $x_j^{+}$ 
            new_c.append(-self.c_orig[j]) # коэффициент при  $x_j^{-}$ 
            for i in range(self.m):
                new_A[i].append(self.A_orig[i][j])
                new_A[i].append(-self.A_orig[i][j])
            new_names.append(f"x_{j+1}+")
            new_names.append(f"x_{j+1}-")
            self.orig_map.append((j, Fraction(1))) #  $x_j^{+}$  даёт  $+x_j$ 
            self.orig_map.append((j, Fraction(-1))) #  $x_j^{-}$  даёт  $-x_j$ 
        else:
            new_c.append(self.c_orig[j])
            for i in range(self.m):
                new_A[i].append(self.A_orig[i][j])
            new_names.append(f"x_{j+1}")
            self.orig_map.append((j, Fraction(1)))

    self.c_orig = new_c
    self.A_orig = new_A
    self.n_new = len(new_c)
    self.var_names_orig = new_names

# -----
# Приведение к каноническому виду
# -----
def _build_canonical(self):
    """Преобразует задачу к виду:  $A x = b, x \geq 0, W \rightarrow \min$ """
    A = []
    b = []
    signs = []

```

```

# копируем строки, исправляя знак b < 0
for i in range(self.m):
    row = list(self.A_orig[i])
    bi = self.b_orig[i]
    sign = self.signs_orig[i]
    if bi < 0:
        row = [-v for v in row]
        bi = -bi
        sign = {'<=': '>=', '>=': '<=', '=': '='}[sign]
    A.append(row)
    b.append(bi)
    signs.append(sign)

# дополнительные переменные для неравенств
extra_count = sum(1 for s in signs if s in ('<=', '>='))
self.n_extra = extra_count

for i in range(self.m):
    row = A[i] + [Fraction(0)] * extra_count
    if signs[i] == '<=':
        k = sum(1 for s in signs[:i] if s in ('<=', '>='))
        row[self.n_new + k] = Fraction(1)
    elif signs[i] == '>=':
        k = sum(1 for s in signs[:i] if s in ('<=', '>='))
        row[self.n_new + k] = Fraction(-1)
    A[i] = row

self.A = A
self.b = b
self.signs = signs
self.c = self.c_orig + [Fraction(0)] * extra_count

# имена переменных: исходные + дополнительные
self.var_names = list(self.var_names_orig)
idx = self.n_new
for s in signs:
    if s in ('<=', '>='):
        idx += 1
    self.var_names.append(f"x{idx}")

self.total_vars = len(self.c)

# ищем начальный базис
self.basis = self._find_initial_basis()

def _find_initial_basis(self):
    """Ищет единичные столбцы для начального базиса"""
    basis = [None] * self.m
    used_rows = set()

    for j in range(self.total_vars):
        col = [self.A[i][j] for i in range(self.m)]
        ones = [i for i, v in enumerate(col) if v == 1]
        others_zero = all(v == 0 or v == 1 for v in col)

```

```

        if len(ones) == 1 and others_zero:
            i = ones[0]
            if i not in used_rows and basis[i] is None:
                basis[i] = j
                used_rows.add(i)

    return basis

# -----
# Вспомогательная задача
# -----
def _build_auxiliary(self):
    """Добавляет искусственные переменные туда, где нет базиса"""
    art_indices = []

    for i in range(self.m):
        if self.basis[i] is None:
            for k in range(self.m):
                self.A[k].append(Fraction(1) if k == i else Fraction(0))
            self.c.append(Fraction(0))
            self.total_vars += 1
            self.var_names.append(f"a{len(art_indices) + 1}")
            self.basis[i] = self.total_vars - 1
            art_indices.append(self.basis[i])

    # W' = сумма искусственных -> min
    aux_c = [Fraction(0)] * self.total_vars
    for j in art_indices:
        aux_c[j] = Fraction(1)

    # выражаем W' через свободные переменные
    aux_b = Fraction(0)
    for i in range(self.m):
        if self.basis[i] in art_indices:
            for j in range(self.total_vars):
                aux_c[j] -= self.A[i][j]
            aux_b -= self.b[i]

    return aux_c, aux_b, art_indices

# -----
# Симплекс-итерации
# -----
def _simplex(self, c, b_offset, basis, A, b, forbid=None):
    """Выполняет симплекс-итерации. Возвращает (status, A, b, c,
    b_offset, basis)"""
    if forbid is None:
        forbid = set()

    A = [row[:] for row in A]
    b = b[:]
    c = c[:]
    basis = list(basis)

    max_iter = 1000

```

```

for _ in range(max_iter):
    # разрешающий столбец
    j_star = None
    min_val = Fraction(0)
    for j in range(self.total_vars):
        if j in forbid:
            continue
        if c[j] < min_val:
            min_val = c[j]
            j_star = j

    if j_star is None:
        return 'optimal', A, b, c, b_offset, basis

    # разрешающая строка
    i_star = None
    best_ratio = None
    for i in range(self.m):
        if A[i][j_star] > 0:
            ratio = b[i] / A[i][j_star]
            if best_ratio is None or ratio < best_ratio:
                best_ratio = ratio
                i_star = i

    if i_star is None:
        return 'unbounded', A, b, c, b_offset, basis

    # пересчёт
    pivot = A[i_star][j_star]
    A[i_star] = [v / pivot for v in A[i_star]]
    b[i_star] = b[i_star] / pivot

    for i in range(self.m):
        if i != i_star and A[i][j_star] != 0:
            factor = A[i][j_star]
            A[i] = [A[i][k] - factor * A[i_star][k] for k in
range(self.total_vars)]
            b[i] = b[i] - factor * b[i_star]

    factor = c[j_star]
    if factor != 0:
        c = [c[k] - factor * A[i_star][k] for k in
range(self.total_vars)]
        b_offset = b_offset - factor * b[i_star]

    basis[i_star] = j_star

    return 'iteration_limit', A, b, c, b_offset, basis

def solve(self):
    """Решает задачу, возвращает (status, x, W)"""
    A = [row[:] for row in self.A]
    b = self.b[:]
    basis = list(self.basis)

```

```

# если базис не полный - решаем вспомогательную задачу
if any(v is None for v in basis):
    aux_c, aux_b, art_indices = self._build_auxiliary()
    A = [row[:] for row in self.A]
    b = self.b[:]
    basis = list(self.basis)

    status, A, b, c_aux, b_aux, basis = self._simplex(
        aux_c, aux_b, basis, A, b
    )

    if status != 'optimal':
        return status, None, None

    if b_aux != 0:
        return 'infeasible', None, None

# убираем искусственные переменные из базиса
for i in range(self.m):
    if basis[i] in art_indices:
        for j in range(self.total_vars):
            if j not in art_indices and A[i][j] != 0:
                pivot = A[i][j]
                A[i] = [v / pivot for v in A[i]]
                b[i] = b[i] / pivot
                for k in range(self.m):
                    if k != i and A[k][j] != 0:
                        f = A[k][j]
                        A[k] = [A[k][t] - f * A[i][t] for t in
range(self.total_vars)]
                        b[k] = b[k] - f * b[i]
                basis[i] = j
                break

        forbid = set(art_indices)
    else:
        forbid = set()

# приводим основную целевую функцию к текущему базису
c = self.c[:]
b_offset = Fraction(0)
for i in range(self.m):
    if basis[i] is not None and c[basis[i]] != 0:
        factor = c[basis[i]]
        c = [c[k] - factor * A[i][k] for k in range(self.total_vars)]
        b_offset = b_offset - factor * b[i]

status, A, b, c, b_offset, basis = self._simplex(
    c, b_offset, basis, A, b, forbid=forbid
)

if status != 'optimal':
    return status, None, None

# значения переменных в расширенной задаче

```

```

        x_ext = [Fraction(0)] * self.total_vars
        for i in range(self.m):
            x_ext[basis[i]] = b[i]

        # восстанавливаем исходные переменные
        x_orig = [Fraction(0)] * self.n
        for j in range(self.n_new):
            orig_idx, sign = self.orig_map[j]
            x_orig[orig_idx] += sign * x_ext[j]

        return 'optimal', x_orig, b_offset

# =====
# Доработал случай свободных переменных
# =====
if __name__ == "__main__":
    print("=" * 60)
    print("Задача по моему варианту 13")
    print("=" * 60)

    # Z = 3x1 + x2 + 2x3 + 4x4 -> max
    # 2x1 + x2 + x3 <= 8
    # x1 + x3 + x4 = 6
    # x2 + x4 >= 4
    c = [3, 1, 2, 4]
    A = [
        [2, 1, 1, 0],
        [1, 0, 1, 1],
        [0, 1, 0, 1],
    ]
    b = [8, 6, 4]
    signs = ['<=', '=', '>=']

    solver = SimplexSolver(c, A, b, signs, maximize=True)
    status, x, W = solver.solve()

    print("Статус:", status)
    if status == 'optimal':
        for i, val in enumerate(x):
            print(f"  x{i+1} = {val}")
        print("Z (max) =", W)

    print()
    print("=" * 60)
    print("Привожу пример что код после правок работает и со свободными
переменными")
    print("=" * 60)

    # Z = x1 + x2 -> max
    # x1 + x2 <= 4
    # x1 - x2 <= 2
    # x1 >= 0, x2 - свободная (любого знака)

    c2 = [1, 1]

```

```
A2 = [  
    [1, 1],  
    [1, -1],  
]  
b2 = [4, 2]  
signs2 = ['<=', '<=']  
  
solver2 = SimplexSolver(c2, A2, b2, signs2, maximize=True, free_vars={1})  
status2, x2, W2 = solver2.solve()  
  
print("Cratyc:", status2)  
if status2 == 'optimal':  
    for i, val in enumerate(x2):  
        print(f" x{i+1} = {val}")  
    print("Z (max) =", W2)
```